

Guide Opérationnel : Automatisation du Trading Haute Performance avec Claude Code et Cowork

1. RÉSUMÉ EXÉCUTIF

L'IA agentique redéfinit radicalement l'ingénierie financière en remplaçant le « vibe coding » (codage intuitif non vérifié) par une rigueur mathématique et logicielle automatisée. Pour un Analyste Quantitatif Senior, Claude Code n'est pas un simple assistant de complétion, mais un environnement de développement autonome capable de gérer le cycle complet : exploration, planification multicouche, exécution de backtests et déploiement. Ce guide détaille comment transformer une thèse de marché en un système de trading 24/7 sécurisé.

Opportunités et Accélération Stratégique :

- **Vitesse de migration** : Capacité démontrée de migrer 10 000 lignes de code Scala vers Java en 4 jours (gain estimé de 10 semaines-homme).
- **Investigation d'incidents** : Réduction de 80 % du temps d'analyse des erreurs système.
- **Optimisation** : Passage du cycle "idée-à-production" de 24 jours à 5 jours en parallélisant les sessions d'agents.

Risques Critiques et Vérité Terrain :

- **Menaces sur la chaîne d'approvisionnement** : L'attaque *axios* (malware nord-coréen) démontre qu'une compromission peut s'exécuter en 2 secondes lors d'un `npm install`.
- **ToxicSkills** : La recherche Snyk a identifié **1 467 payloads malveillants** dans l'écosystème des skills IA, avec un taux de failles de sécurité de 24 à 45 % dans le code généré.
- **Saturation de contexte** : Performance dégradée au-delà de 200 000 jetons (tokens), nécessitant une gestion agressive via `/compact`.

Conditions Impératives de Mise en Production :

1. **Validation Paper Trading** : Observation du comportement réel sans exposition au capital.
2. **Audit Snyk & Isolation** : Scan obligatoire des dépendances et exécution dans des environnements éphémères (Orbstack/Dev Containers) pour protéger les clés SSH.
3. **Certification des Backtests** : Analyse de robustesse via QuantStats (30+ métriques institutionnelles).

2. ANALYSE THÉMATIQUE DES SOURCES

La robustesse d'un système financier dépend de la « Ground Truth » technique. Le tableau suivant synthétise les éléments exploitables pour un architecte logiciel.

Thème	Concept Clé	Élément Exploitable	Limites & Gestion
Infrastructure	CBT Framework	21 commandes (ex: <code>/cbt:optimize</code>)	Requiert Node.js 16+ et Python 3.8+.
Architecture	Trois Routes de Données	Latence : 10ms (DuckDB) vs 60s+ (API)	Prioriser Historify/DuckDB pour la performance.
Sécurité	Isolation Bash Tool	Sandboxing via <code>/sandbox</code> ou VM	Bloquer l'accès aux clés SSH lors des installs.
Méthodologie	Modèle Writer/Reviewer	Session A (Code) / Session B (Audit)	Évite le biais de confirmation de l'agent.
Context Mgmt	Compaction / Rewind	<code>/compact</code> et menu <code>/rewind</code>	Nécessaire pour rester sous la limite de 200k.

Information manquante : Les sources ne fournissent pas de connecteurs pré-configurés pour les flux temps réel d'Euronext Paris ; l'implémentation devra s'appuyer sur des serveurs MCP personnalisés (ex: Alpha Vantage).

3. CONCEPTS FONDAMENTAUX DU TRADING VIA CLAUDE

En tant qu'architecte, la précision terminologique est votre première ligne de défense contre les erreurs de logique financière.

Indicateurs et Prévention du Biais de Lookahead

Le "lookahead bias" (biais d'anticipation) est le risque majeur en trading algorithmique. Pour le neutraliser, toute logique de signal doit impérativement utiliser `.shift(1)`.

- **Règle Institutionnelle** : Ne jamais utiliser les fonctions intégrées de VectorBT pour générer des indicateurs (EMA, RSI, VWAP). Utilisez exclusivement **TA-Lib** pour garantir des résultats de grade professionnel et éviter le "Vibe Coding".

Gestion du Risque et "Kill Switch"

L'automatisation exige un découplage entre la stratégie et le contrôle des risques :

- **Sizing Granulaire** : Pour les actions, configurer `min_size=1` et `size_granularity=1` afin d'éviter les rejets d'ordres sur les plateformes ne supportant pas les fractions de titres.
- **Drawdown Kill Switch** : Un script indépendant doit surveiller le solde API et révoquer les permissions de trading si la perte maximale autorisée est atteinte.

Hierarchie des Tests

1. **Backtesting** : Historique pur.
2. **Robustesse (Monte Carlo)** : Simulation de 1 000 variantes de marché pour valider la stabilité du Sharpe Ratio.
3. **Paper Trading** : Test de latence et de slippage (frais réels : 0,111% actions indiennes vs 0,1% crypto spot).

4. STRATÉGIES DE TRADING IDENTIFIÉES

Le choix de la stratégie doit précéder toute ligne de code. L'erreur fatale est de "tirer une stratégie de nulle part" sans fondement théorique.

1. Scalping (Source : Lewis Jackson)

- **Logique** : Prix > VWAP, Prix > EMA 8, RSI < 30.
- **Actif** : XRP/USDT (Timeframe 1 min).
- **Note technique** : Nécessite une exécution ultra-rapide (Route 1 : DuckDB).

2. EMA Crossover (Source : VectorBT Skills)

- **Logique** : Croisement 10/30 EMA sur State Bank of India (SBIN).
- **Note technique** : Idéal pour une première automatisation via `/cbt:build`.

3. Earnings Credit Spreads (Source : Reddit/mastagoose)

- **Logique** : Vente d'options (Call Credit Spreads) juste avant les résultats financiers.
- **Note technique** : Utiliser Claude pour analyser le sentiment et le score de confiance avant l'entrée.

5. LOGIQUE PROGRAMMABLE ET PSEUDO-CODE ARCHITECTURAL

L'utilisation du pseudo-code permet d'économiser les jetons de contexte et de valider la structure avant la génération.

```
// PARAMÈTRES ARCHITECTURAUX
CONST SYMBOL = "BTC/USDT"
CONST MIN_SIZE = 1           // Contrainte de lot entier
CONST SIZE_GRANULARITY = 1
CONST RISK_PER_TRADE = 0.02 // 2% du capital
```

```
// LOGIQUE DE SÉCURITÉ
PROCÉDURE Verification_Securite:
    SI Scan_Snyk_Vulnerabilites > 0 ALORS ABORT
    SI Connexion_Echange == FALSE ALORS NOTIFY_ADMIN & RETRY
    SI Solde_Disponible < (PRICE * MIN_SIZE) ALORS STOP_BOT

// BOUCLE D'EXÉCUTION (Latence cible < 50ms)
BOUCLE Temps_Réel:
    DONNÉES = Route_2_OpenAlgo_DB() // Historify DuckDB
    EMA_SHORT = TA_Lib.EMA(DONNÉES, 10).shift(1)
    EMA_LONG = TA_Lib.EMA(DONNÉES, 30).shift(1)

    SI EMA_SHORT CROSSES_ABOVE EMA_LONG:
        TAILLE = Calculer_Position(Capital, SL_Dist)
        SI TAILLE >= MIN_SIZE:
            EXECUTE_ORDER(BUY, ROUND(TAILLE, SIZE_GRANULARITY))

    SI DRAWDOWN > MAX_ALLOWED:
        ACTIVER_KILL_SWITCH() // Fermeture immédiate des positions
```

Cas limites (Edge Cases) : Expiration des credentials AWS (utiliser un script "keepalive"), latence réseau supérieure à 2s, ou déconnexion du WebSocket de l'échange.

6. ARCHITECTURE CIBLE DU SYSTÈME

Un architecte doit concevoir pour la persistance et la performance.

- **Data Tier** : Utilisation de **DuckDB (Historify)**. Latence critique : Route 1 (10ms) pour le trading haute fréquence, Route 3 (60s+) réservée aux analyses macro.
 - **Logic Tier** : **CBT Framework** (Fast Engine via Polars/Numba). Pas de pandas dans le chemin critique d'exécution.
 - **MCP Integration** : Injecter des données macro via les serveurs **Alpha Vantage** ou **FRED** pour corrélérer les signaux techniques avec l'inflation ou les taux de la Fed.
 - **Execution Tier** : Déploiement sur **Railway**. *Justification* : Les cycles de veille des ordinateurs locaux interrompent les connexions WebSocket ; Railway assure une persistance 24/7.
 - **Audit Trail** : Journalisation automatique (Accounting Log) pour les obligations fiscales.
-

7. PLAN D'OPÉRATIONNALISATION AVEC CLAUDE CODE

Adoptez la méthode : "**Explore first, then plan, then code**".

1. **Onboarding Agent** : Utilisez le prompt "Master One-Shot" de Lewis Jackson pour configurer l'environnement en 5 minutes (Clonage, API keys, .env).
2. **Gestion de Session** :
 - **/btw** : Pour les questions rapides ne nécessitant pas de polluer le contexte.

- `/rewind` : Pour annuler des expérimentations algorithmiques infructueuses sans perdre l'état du code.
 - `/compact` : À exécuter dès que la saturation approche pour résumer les décisions clés.
3. **Commandes CBT** : Exploitez les 21 commandes, notamment `/cbt:research` pour valider la littérature financière et `/cbt:optimize` pour le walk-forward analysis.
-

8. COORDINATION AVEC CLAUDE COWORK

Utilisez Cowork comme votre PM et QA Engine pour superviser la flotte d'agents.

- **Pattern Writer/Reviewer** : La Session A écrit la logique de trading ; la Session B (persona `security-reviewer.md`) l'audite. Un agent frais est moins biaisé.
 - **Devil's Advocate Mode** : Forcez Cowork à remettre en question votre stratégie (ex: "Simule une hausse de 100% de la volatilité VIX, ma gestion du risque survit-elle ?").
 - **Fractal PM** : Décomposez les objectifs complexes en "prédicats vérifiables" pour éviter que Claude ne se perde dans la complexité du code.
-

9. PRODUIT MINIMUM VIABLE (MVP) SÉCURISÉ

Le MVP doit prouver la viabilité sans exposer un centime.

- **Infrastructure** : Isolation via `/sandbox` ou Orbstack pour prévenir toute exécution de malware post-install (axios).
 - **Périmètre** : 1 actif (BTC), données 1 an, mode Paper Trading.
 - **Livrable** : Une "Tearsheet QuantStats" avec score de confiance IA.
 - **Validation Sécurité** : Zéro vulnérabilité Snyk détectée dans le répertoire `.claude/skills/`.
-

10. ROADMAP ET PLAN D'ACTION (30 JOURS)

- **Jours 1-7 : Installation & Audit.** Déploiement de Claude Code, configuration MCP (Alpha Vantage), et scan de sécurité des 1 467 menaces potentielles via Snyk.
 - **Jours 8-14 : Développement Quant.** Implémentation TA-Lib, backtests via `/cbt:run`, et optimisation du sizing (`min_size=1`).
 - **Jours 15-21 : Persistance Cloud.** Migration vers Railway et monitoring des premières transactions virtuelles.
 - **Jours 22-30 : Scaling Contrôlé.** Passage en capital réel limité (ex: 500\$) avec un Kill Switch de drawdown strict activé.
-

ANNEXES

Glossaire Technique :

- **Lookahead Bias** : Erreur utilisant des données futures pour prédire le passé. Neutralisé par `.shift(1)`.
- **MCP (Model Context Protocol)** : Standard permettant à Claude d'accéder à des outils externes (Alpha Vantage, FRED).
- **Slippage** : Écart entre le prix demandé et le prix d'exécution.
- **Agentic Loop** : Cycle autonome où l'IA planifie, exécute et s'auto-corrige.

Modèle de README.md (Projet) :

```
# AlphaQuant Bot v1.0
- Engine: CBT Framework (Polars/Numba)
- Data Route: Route 1 (DuckDB/10ms)
- Indicators: TA-Lib (Shifted)
- Security: Snyk Audited / Orbstack Isolated
- Risk: Whole-share sizing (min_size=1)
```

Journal de Trading Type : | Date | Stratégie | Signal | Résultat | Note de l'IA (Confidence) | | :--- | :--- | :--- | :--- | | 2026-06-01 | EMA Cross | Long | +1.5% | 85% (Sentiment Macro Positif via FRED) | | 2026-06-02 | Scalping | Blocked | N/A | Risk Management: RSI non atteint (<30) |

AVERTISSEMENT FINAL : Le trading algorithmique comporte des risques de perte totale du capital. L'IA peut halluciner des opportunités ou introduire des failles de sécurité. La responsabilité finale de l'audit du code et de la gestion des fonds incombe exclusivement à l'utilisateur humain. Information non disponible dans les sources concernant les garanties de profit.